BREAST CANCER PREDICTION

Using Logistic Regression

A Machine Learning Project Documentation

This document explains, step by step, how the Breast Cancer Wisconsin (Diagnostic) dataset was analyzed and how a Logistic Regression model was built, evaluated, and deployed through a Streamlit web application. It includes the dataset overview, data cleaning and preprocessing, exploratory data analysis, feature engineering, model training, performance evaluation, and deployment.

1. Project Title

Breast Cancer Prediction using Logistic Regression — a binary classification project that predicts whether a breast tumor is Malignant (M) or Benign (B) from cell-nuclei measurements extracted from digitized images of a fine needle aspirate (FNA).

2. Dataset Size

- Source: Breast Cancer Wisconsin (Diagnostic) Dataset, obtained from Kaggle.
- Size: 569 rows × 33 columns, as loaded from data.csv.
- 30 real-valued predictive features
- 1 identifier column (id)
- 1 target column (diagnosis)
- 1 empty artifact column (Unnamed: 32) — dropped during cleaning
- Class distribution: Benign cases outnumber Malignant cases — a mild class imbalance.

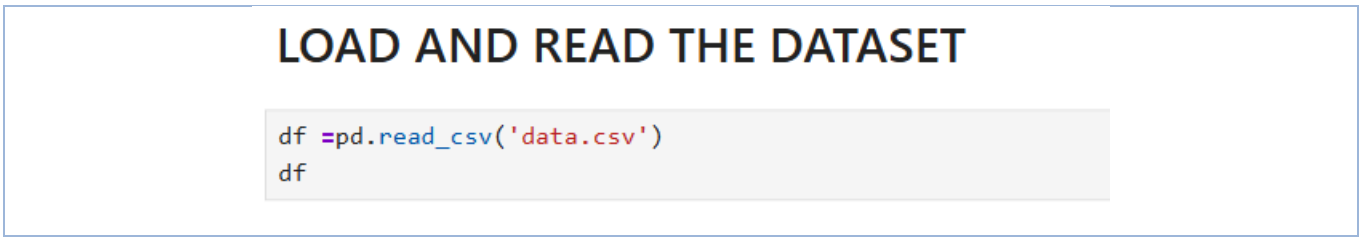


Figure 1: Loading the dataset and inspecting

```
df.shape
```

```
(569, 33)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #    Column                   Non-Null Count  Dtype  
---   ------                   --------------  -----  
 0    id                       569 non-null    int64  
 1    diagnosis                569 non-null    object 
 2    radius_mean              569 non-null    float64
 3    texture_mean             569 non-null    float64
 4    perimeter_mean           569 non-null    float64
 5    area_mean                569 non-null    float64
 6    smoothness_mean          569 non-null    float64
 7    compactness_mean         569 non-null    float64
 8    concavity_mean           569 non-null    float64
 9    concave points_mean      569 non-null    float64
 10   symmetry_mean            569 non-null    float64
 11   fractal_dimension_mean   569 non-null    float64
 12   radius_se                569 non-null    float64
 13   texture_se               569 non-null    float64
 14   perimeter_se             569 non-null    float64
 15   area_se                  569 non-null    float64
 16   smoothness_se            569 non-null    float64
 17   compactness_se           569 non-null    float64
 18   concavity_se             569 non-null    float64
 19   concave points_se        569 non-null    float64
 20   symmetry_se              569 non-null    float64
 21   fractal_dimension_se     569 non-null    float64
 22   radius_worst             569 non-null    float64
 23   texture_worst            569 non-null    float64
 24   perimeter_worst          569 non-null    float64
 25   area_worst               569 non-null    float64
 26   smoothness_worst         569 non-null    float64
 27   compactness_worst        569 non-null    float64
 28   concavity_worst          569 non-null    float64
 29   concave points_worst     569 non-null    float64
 30   symmetry_worst           569 non-null    float64
 31   fractal_dimension_worst  569 non-null    float64
 32   Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

its shape/structure.

3. Feature Names

30 numeric features, each computed three ways (mean, standard error — "_se", and worst/largest — "_worst") for 10 underlying cell-nucleus measurements:

```
df.columns
```

```
Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst', 'Unnamed: 32'],
      dtype='object')
```

4. Feature Datatypes

- id → int64 (identifier, dropped)
- diagnosis → object, later encoded to int (0/1) as the target
- 30 measurement columns → float64 (e.g., radius_mean, area_worst)
- Unnamed: 32 → float64, but 100% missing values, dropped

5. EDA Graphs & Observations

Histograms

area_mean, area_worst, radius_mean, radius_worst, perimeter_mean, perimeter_worst are right-skewed; several other features are roughly normally distributed.

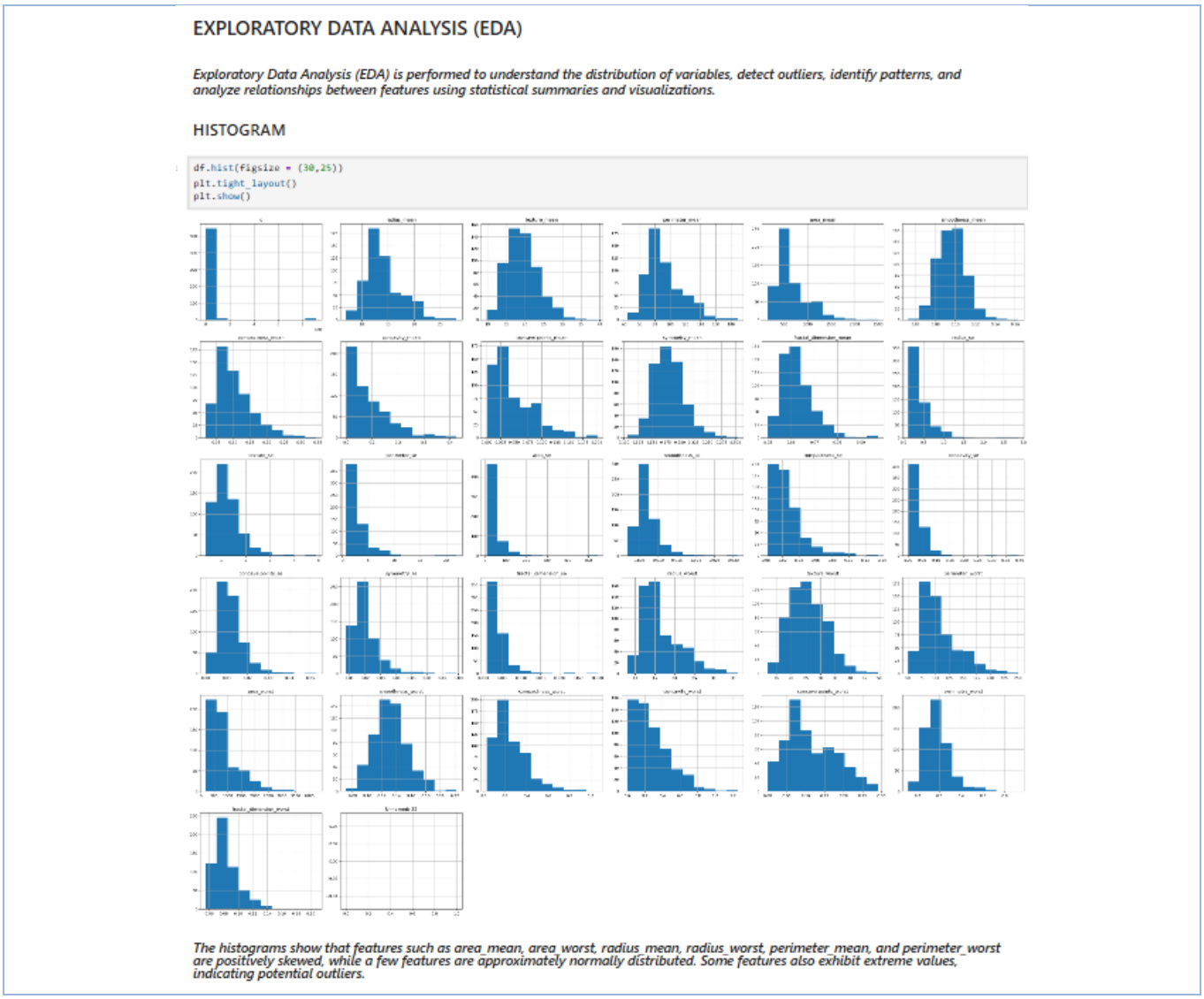


Figure 2: Distribution of all numeric features after cleaning.

Scatterplots

radius_mean vs perimeter_mean, radius_mean vs area_mean, concavity_mean vs concave points_mean: strong positive linear relationships between size-related features, and malignant/benign points form visually separable clusters — a good early signal these features will be predictive.

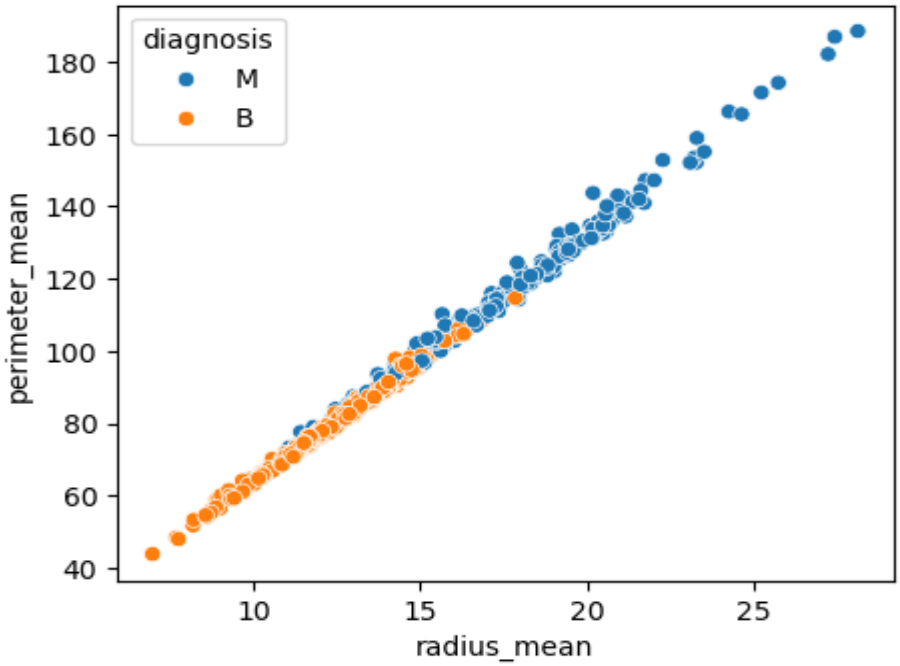


Figure 3: radius_mean vs perimeter_mean, colored by diagnosis — classes separate fairly well.

Count Plot

Benign cases are more frequent than Malignant, confirming mild class imbalance.

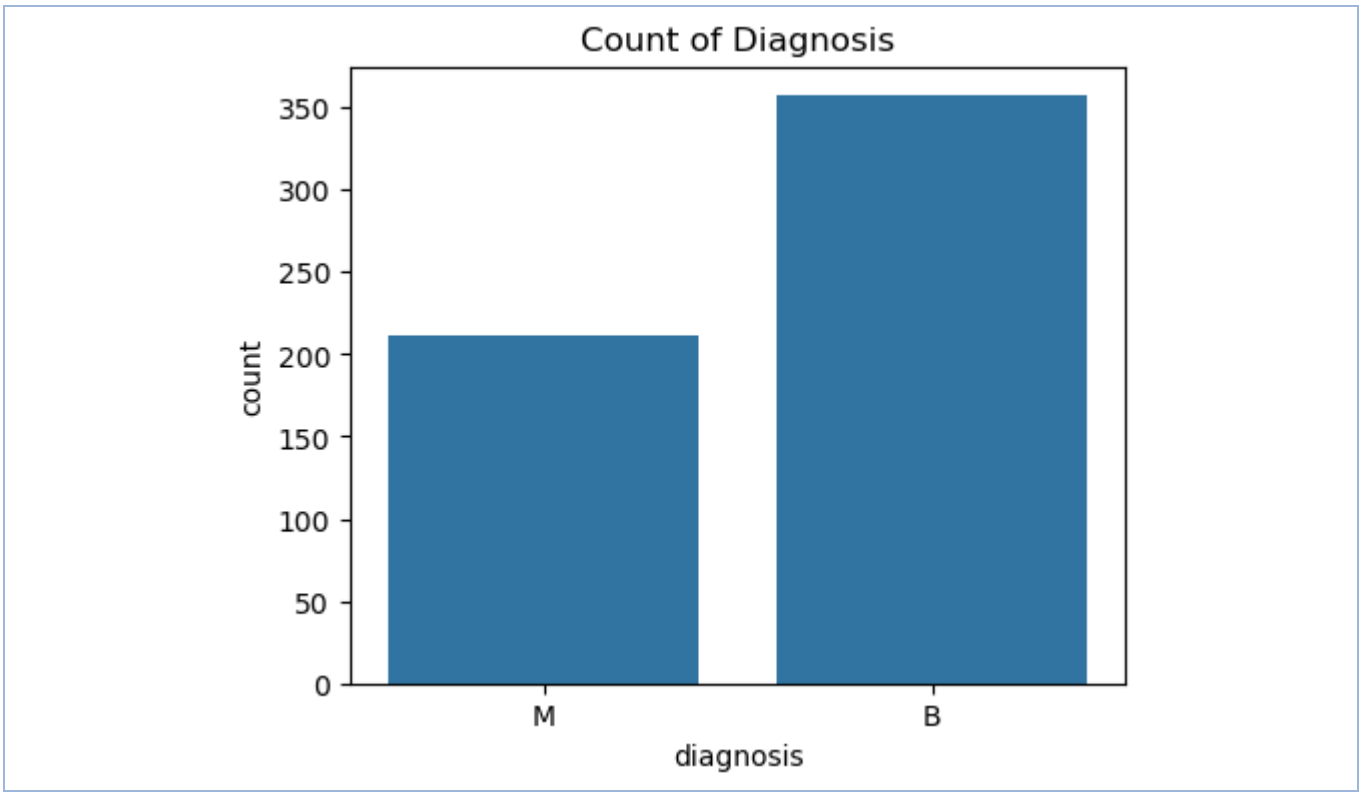


Figure 4: Count of Benign vs Malignant diagnoses.

6. Outliers — Why They Occur & How They Were Handled

Why outliers occur

Malignant tumors are biologically larger, more irregular, and more variable in shape than benign ones. This produces genuinely extreme values (large radius, area, concavity) in the malignant class — real biological signal, not data-entry errors. This shows up clearly as right-skew in the histograms and as extreme points in the boxplots below.

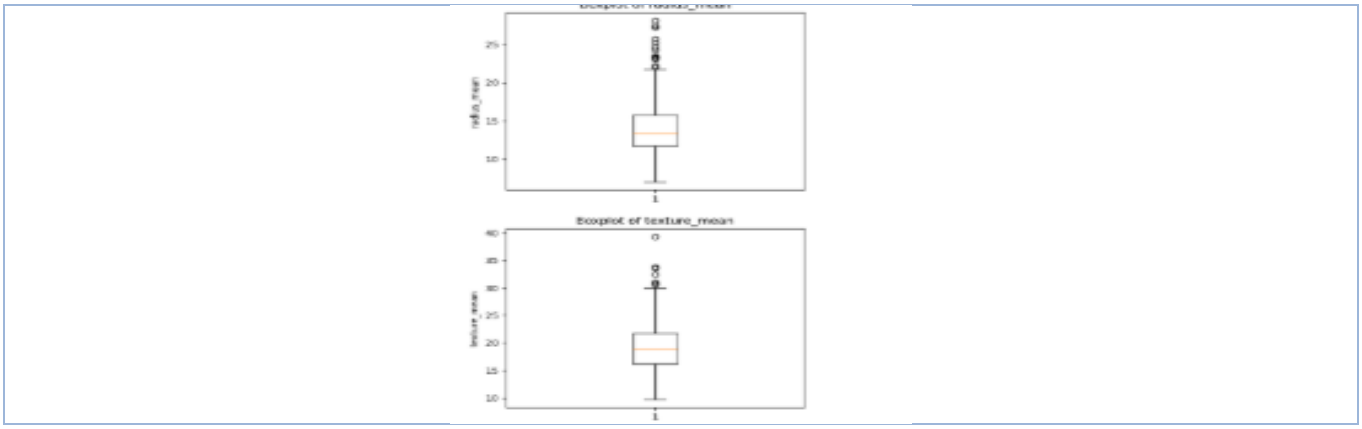


Figure 5: Boxplot of radius_mean before outlier treatment, showing points beyond the whiskers.

Why they need to be handled

Logistic Regression fits a linear decision boundary using gradient-based optimization; a few extreme values can disproportionately pull the coefficients and distort the boundary. Left untreated, this can hurt generalization to new data.

7. Null Values — How & Why Replaced

- df.isnull().sum() showed zero missing values in all 30 real feature columns.
- The only column with nulls was Unnamed: 32, which was 100% null (569/569) — a byproduct of a trailing comma in the Kaggle CSV export, not real data.
- Handling: No imputation was needed for the real features. Unnamed: 32 was dropped entirely (not imputed) since it carries zero information; id was also dropped since it's a unique identifier with no predictive value.

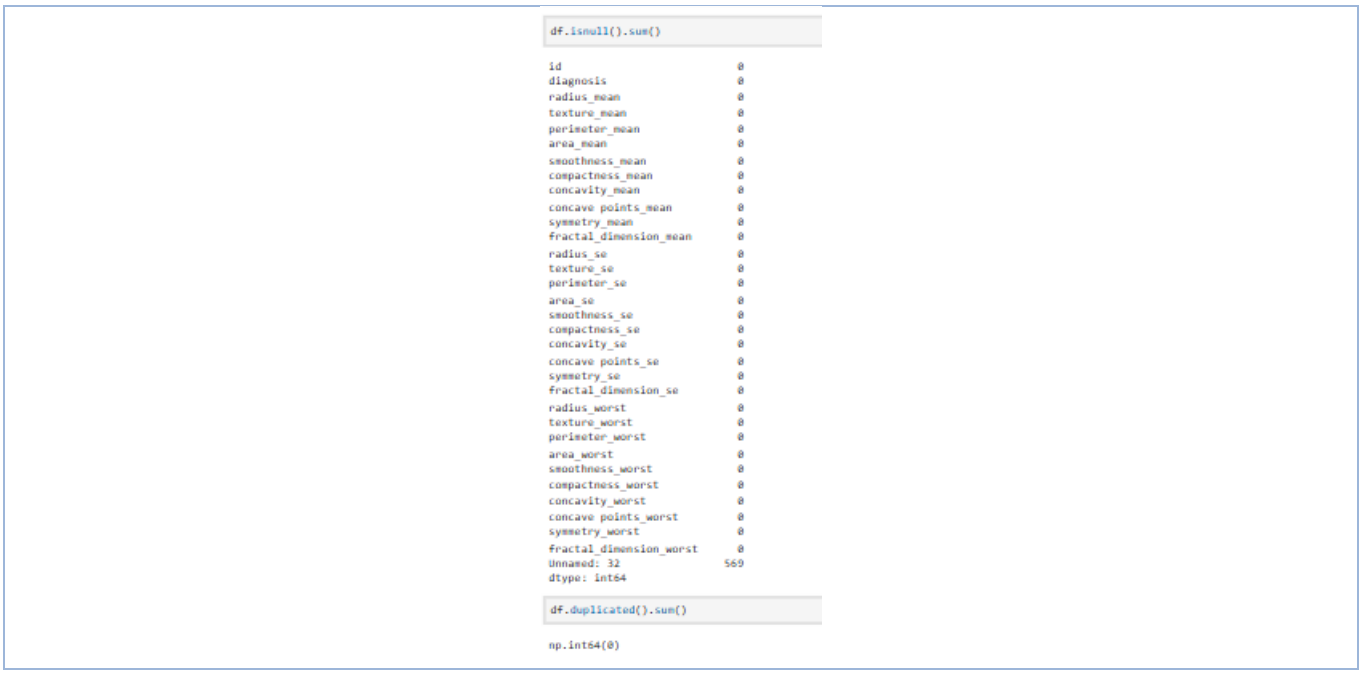
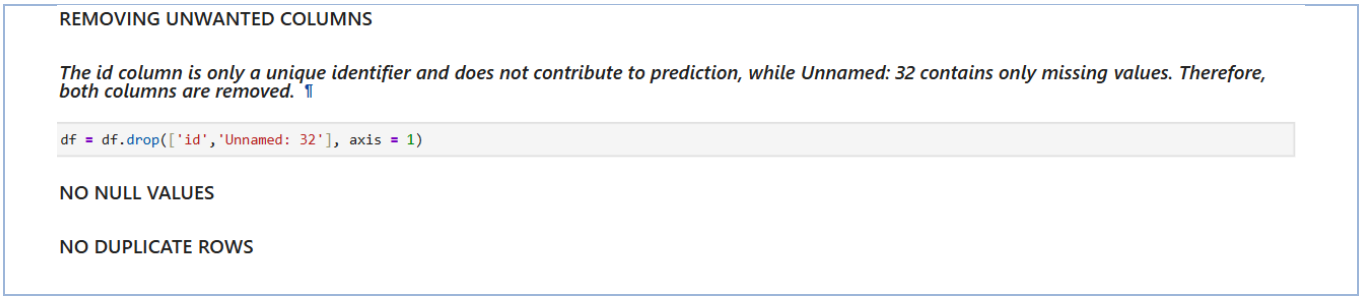


Figure 6: Null-value check and removal of non-informative columns.

8. Data Cleaning

- Dropped id and Unnamed: 32 columns.
- Confirmed 0 duplicate rows using df.duplicated().sum().
- IQR-clipped outliers on all numeric columns.
- Encoded diagnosis: M → 1, B → 0.



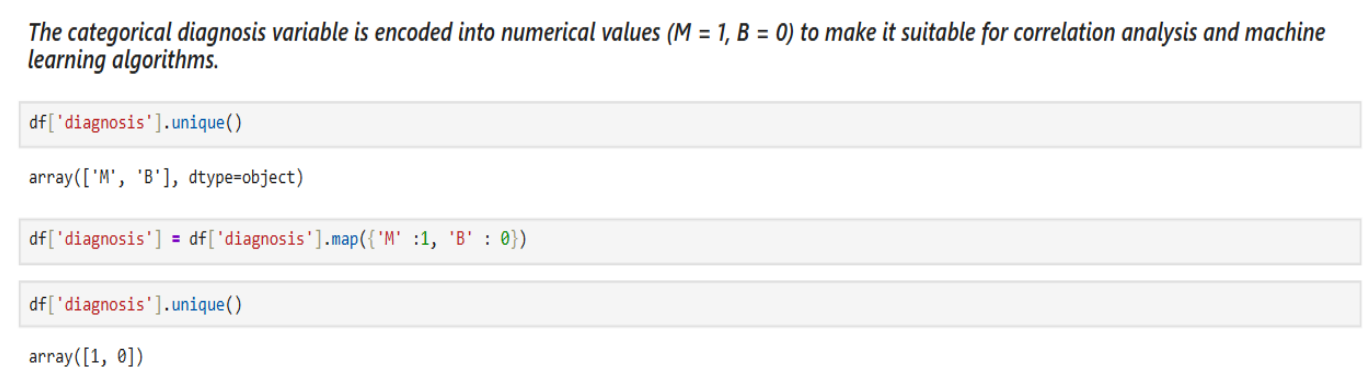


Figure 7: Encoding the diagnosis column into numeric labels (M=1, B=0).

HANDLING OUTLIERS

Outliers are treated using the IQR method with clipping to reduce the impact of extreme values while preserving all observations.

```
num_cols = df.select_dtypes(include='number').columns

for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df[col] = df[col].clip(lower=lower, upper=upper)
```

Outliers have been capped within the IQR limits, reducing their influence on the dataset and improving its suitability for analysis and model training.

Figure 8: IQR-based clipping applied to all numeric columns.

IQR-based clipping/capping was used instead of deleting rows.

Why clipping instead of removing rows

The dataset only has 569 rows — deleting rows would shrink an already-small dataset and could disproportionately remove malignant cases (since they produce more extreme values), worsening class imbalance and losing signal. Clipping keeps every observation but limits the influence of extreme points.

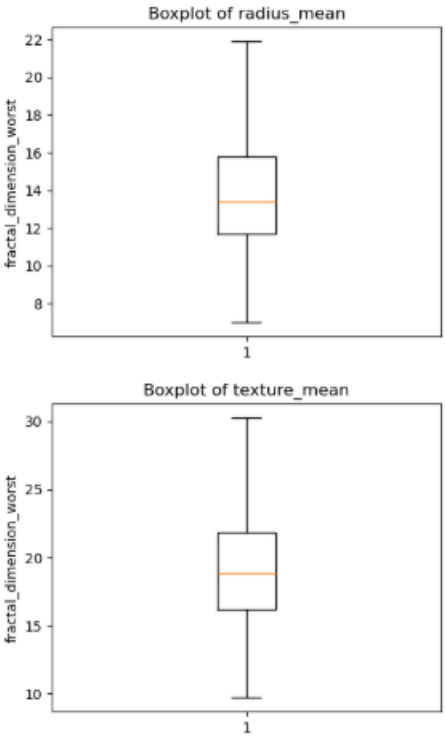
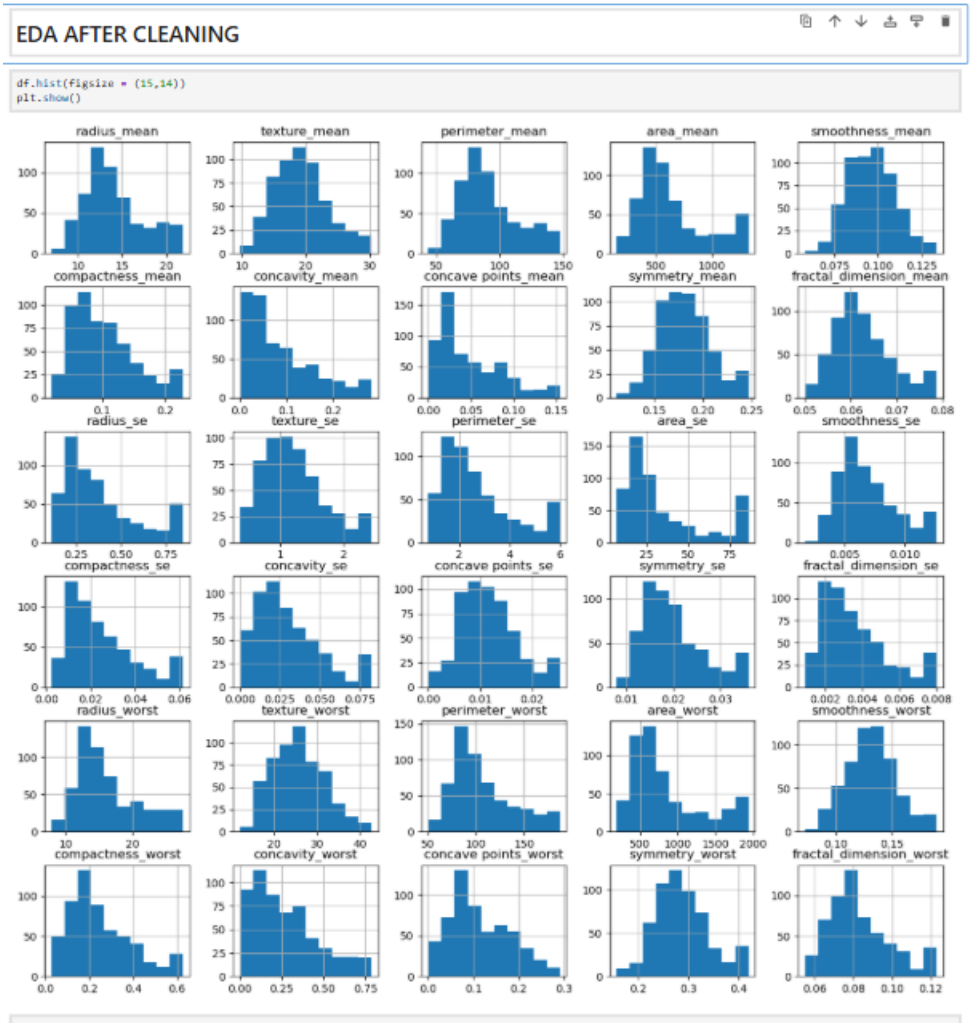
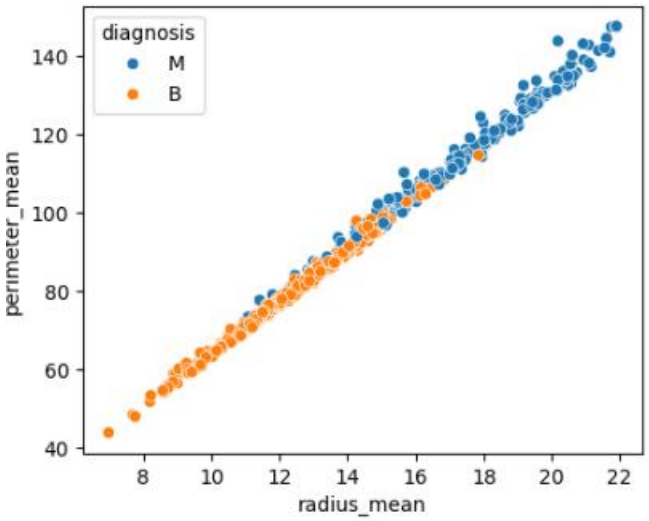
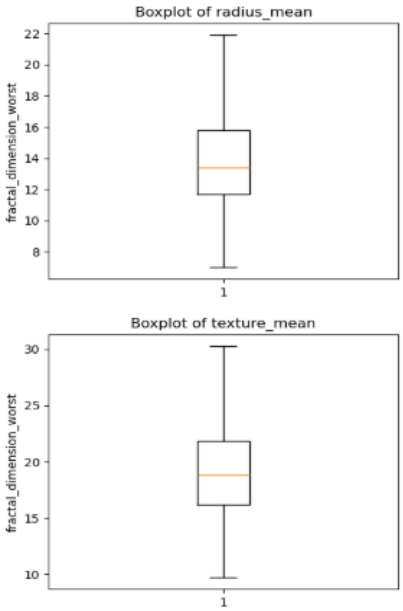


Figure 9: Boxplot of the same feature after IQR clipping — extreme points are pulled in.


EDA AFTER CLEANING
df.hist(figsize = (15,14))
plt.show()
radius_mean
texture_mean
perimeter_mean
area_mean
smoothness_mean
compactness_mean
concavity_mean
concave points_mean
symmetry_mean
fractal_dimension_mean
radius_se
texture_se
perimeter_se
area_se
smoothness_se
compactness_se
concavity_se
concave points_se
symmetry_se
fractal_dimension_se
radius_worst
texture_worst
perimeter_worst
area_worst
smoothness_worst
compactness_worst
concavity_worst
concave points_worst
symmetry_worst
fractal_dimension_worst


diagnosis
M
B
perimeter_mean
radius_mean


Boxplot of radius_mean
fractal_dimension_worst
Boxplot of texture_mean
fractal_dimension_worst

9. Correlation Analysis:

A correlation matrix is generated to measure the relationship between numerical features and the target variable, helping identify the most relevant features for prediction.

```
df.corr()
```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
diagnosis	1.000000	0.745217	0.419123	0.759006	0.752786	0.366931	0.608494	0.727168	0.7880
radius_mean	0.745217	1.000000	0.332760	0.997692	0.993891	0.167672	0.502741	0.674539	0.8118
texture_mean	0.419123	0.332760	1.000000	0.339588	0.341552	-0.015997	0.241666	0.318654	0.3015
perimeter_mean	0.759006	0.997692	0.339588	1.000000	0.991861	0.204990	0.553698	0.714624	0.8418
area_mean	0.752786	0.993891	0.341552	0.991861	1.000000	0.167400	0.498665	0.680126	0.8141
smoothness_mean	0.366931	0.167672	-0.015997	0.204990	0.167400	1.000000	0.661938	0.526544	0.5609

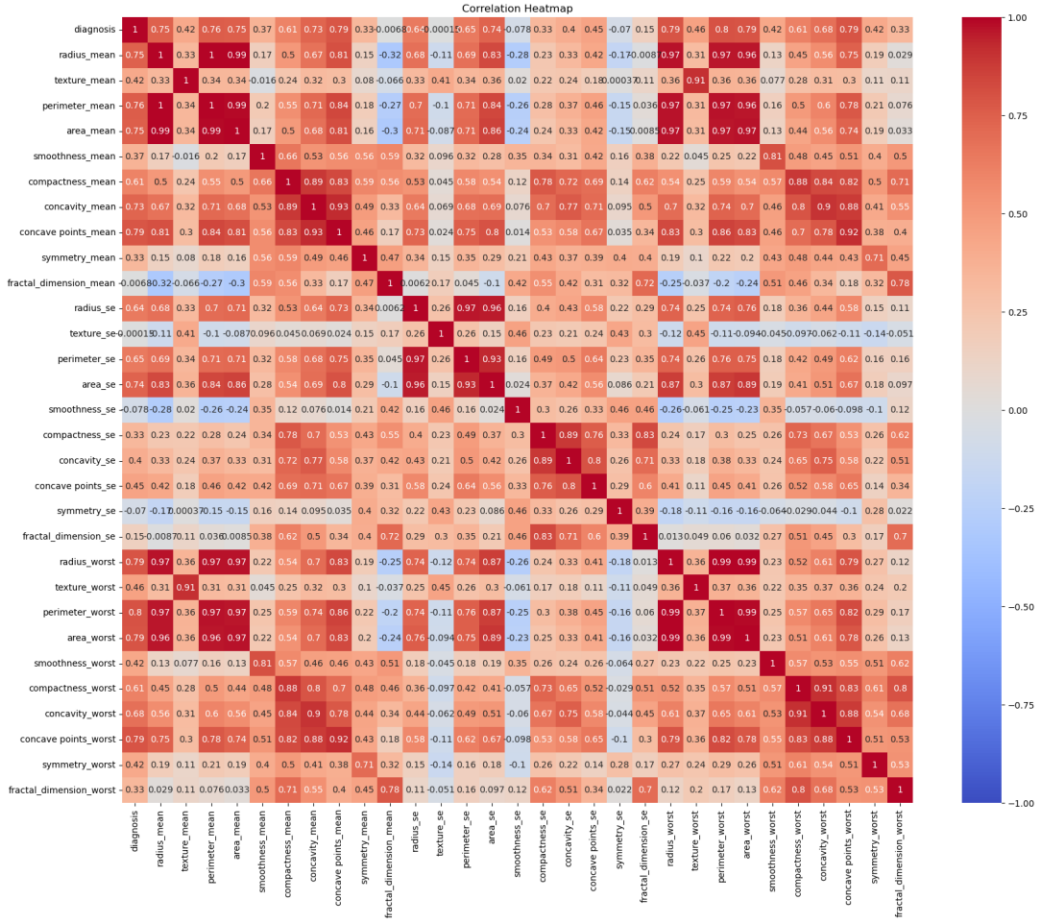
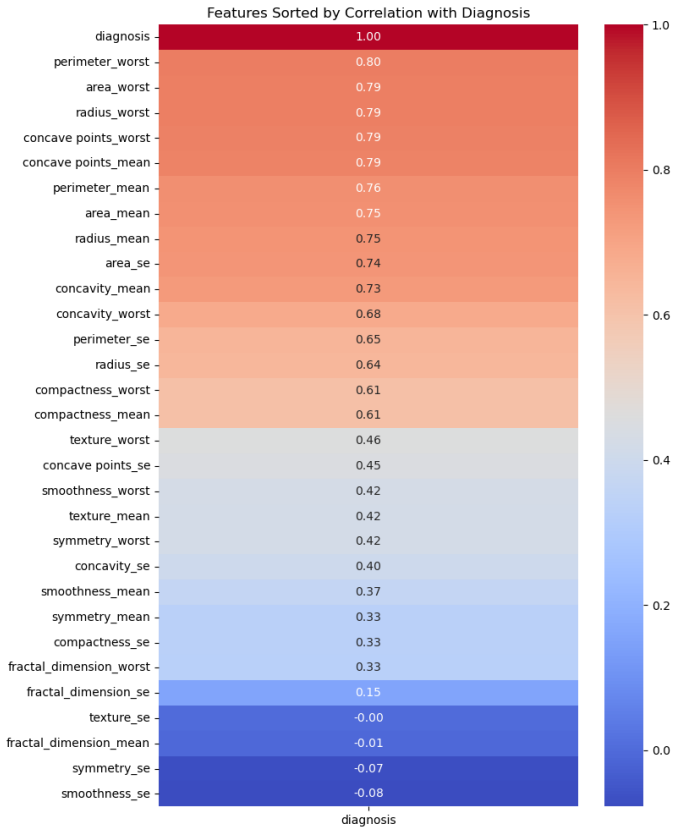


Figure 10: Correlation heatmap of all numeric features.

Correlation Heatmap

radius, perimeter, area, and concavity-related features are strongly inter-correlated (expected, since they're geometrically related) and strongly correlated with diagnosis.



10. Selected Features:

The selected features are assigned to X (independent variables), and the diagnosis column is assigned to y (target variable)

```
X = df[['perimeter_worst','area_worst','radius_worst','concave points_worst','concave poin
y = df['diagnosis']
```

```
X.columns
```

```
Index(['perimeter_worst', 'area_worst', 'radius_worst', 'concave points_worst',
       'concave points_mean', 'perimeter_mean', 'area_mean', 'radius_mean',
       'area_se', 'concavity_mean', 'concavity_worst', 'perimeter_se',
       'radius_se', 'compactness_worst', 'compactness_mean'],
      dtype='object')
```

```
X.shape
```

```
(569, 15)
```

Figure 11: Selecting the 15 features most correlated with the diagnosis target.

Target column: diagnosis (M = Malignant, B = Benign).

- **Why:** Logistic Regression builds a linear decision boundary, so features with the strongest linear relationship to the target carry the most useful signal; dropping weakly-correlated features reduces noise and dimensionality.
- **Caveat:** Several selected features are highly collinear with each other (radius/perimeter/area are all derived from the same tumor geometry). This doesn't hurt raw predictive accuracy much, but it does inflate the variance of individual coefficients, making them less interpretable individually. Since the goal here is prediction accuracy (not coefficient interpretation), this is an acceptable simplification.

11. Data Transformation (Feature Scaling)

StandardScaler was applied to all 15 selected features (zero mean, unit variance).

STANDARD SCALER

StandardScaler is used to standardize numerical features to a common scale for better model performance.

```
#FOR NUMERIC
SS=StandardScaler()
SS_X=SS.fit_transform(X)
SS_X=pd.DataFrame(SS_X)
SS_X.columns=['perimeter_worst','area_worst','radius_worst','concave points_worst','concave points_mean','perimeter_mean','area_mean','radius_mean','are
SS_X.head()
```

	perimeter_worst	area_worst	radius_worst	concave points_worst	concave points_mean	perimeter_mean	area_mean	radius_mean	area_se	concavity_mean	concavity_worst	perin
0	2.439568	2.287627	2.006477	2.296076	2.620973	1.357375	1.184085	1.176800	2.110995	2.647422	2.246192	2
1	1.631542	2.287627	1.921384	1.087084	0.574944	1.795991	2.249396	1.949929	1.611678	-0.000497	-0.137634	0
2	1.434234	1.807751	1.611558	1.955000	2.110330	1.670052	1.846217	1.686226	2.110995	1.496076	0.920718	1
3	-0.245395	-0.593838	-0.277945	2.175786	1.506601	-0.606410	-0.831485	-0.791983	-0.318438	2.091997	2.119474	0
4	1.424838	1.525780	1.386825	0.729259	1.482665	1.891531	2.154338	1.866023	2.110995	1.504202	0.665254	1

Figure 12: Feature scaling with StandardScaler, followed by the stratified train/test split.

Why: features are on very different scales (e.g., area_mean ranges into the hundreds/thousands, while concavity_mean ranges 0–0.4). Logistic Regression's gradient-based solver converges faster and treats all features fairly when they're on a common scale; without scaling, large-magnitude features would dominate the loss function regardless of true importance.

12. Data Partition & Ratio

DATA PARTITION

```
X_train, X_test, Y_train, Y_test = train_test_split(SS_X,y,test_size=0.3,random_state=42, stratify = y)
```

The dataset is split into training (70%) and testing (30%) sets to train the model and evaluate its performance on unseen data. ¶

- 70% train / 30% test → 398 training rows / 171 test rows.
- random_state=42 used for reproducibility.
- stratify=y preserves the same Benign/Malignant ratio in both splits, which matters given the class imbalance and the relatively small 171-row test set.

13. Model Selection

Logistic Regression was chosen because:

- The task is binary classification (Malignant vs Benign) — logistic regression is the natural baseline.
- EDA and feature analysis indicated that the selected features provide good class separation, making Logistic Regression an appropriate baseline model for this classification task.
- It is interpretable (coefficients relate to log-odds), fast to train, and works well on small-to-medium tabular datasets like this one (569 rows).
- It is a standard, well-validated baseline for medical diagnostic classification problems.

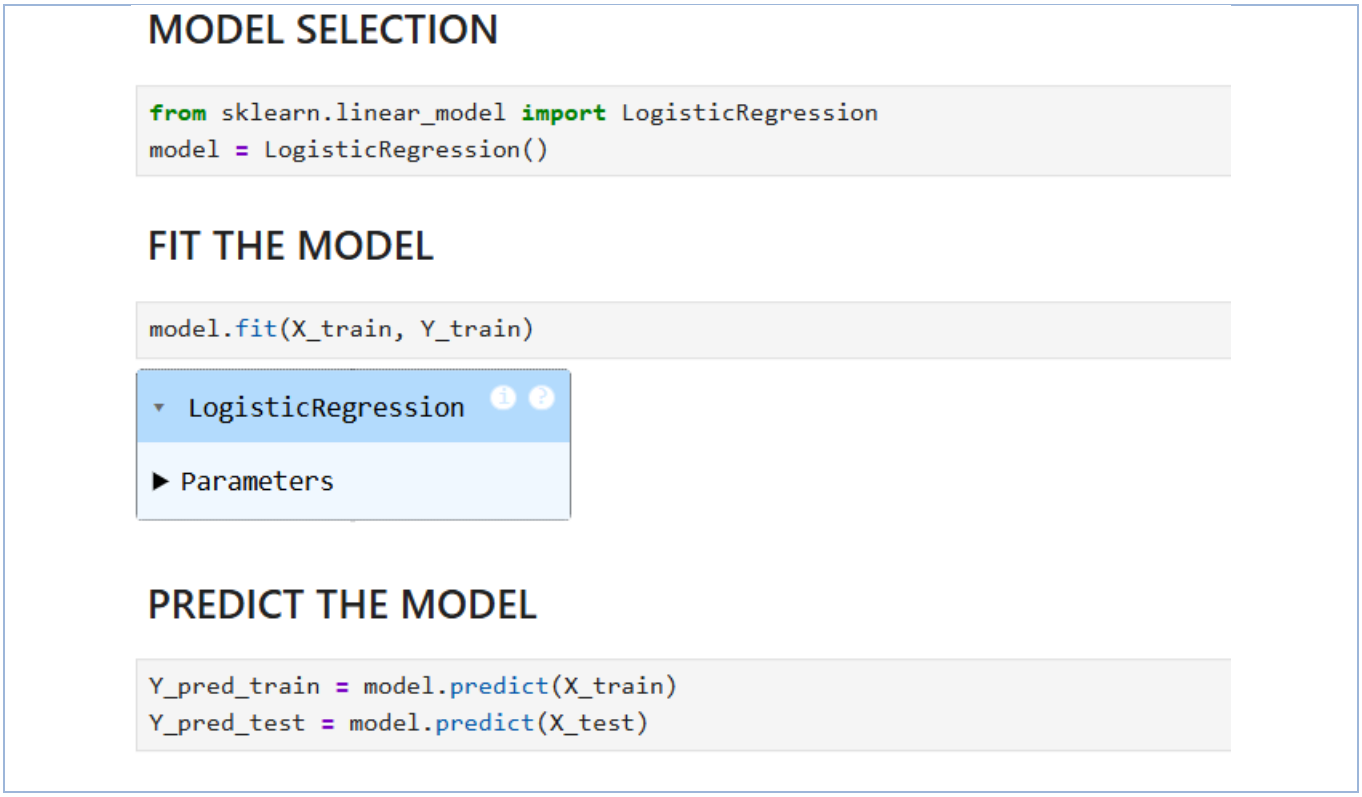


Figure 13: Training the Logistic Regression model.

The Logistic Regression model was trained using the fit() method on the training dataset. During training, the model learns the relationship between the selected features and the target variable. The trained model is then used to predict whether a tumor is **Malignant (M)** or **Benign (B)** for new unseen data.

MODEL EVALUALUATION

```
train_accuracy = accuracy_score(Y_train,Y_pred_train)
train_precision = precision_score(Y_train,Y_pred_train)
train_recall = recall_score(Y_train,Y_pred_train)
train_f1 = f1_score(Y_train,Y_pred_train)
test_accuracy = accuracy_score(Y_test,Y_pred_test)
test_precision = precision_score(Y_test,Y_pred_test)
test_recall = recall_score(Y_test,Y_pred_test)
test_f1 = f1_score(Y_test,Y_pred_test)
```

```
print("Training Accuracy :", np.round(train_accuracy,2))
print("Testing Accuracy  :", np.round(test_accuracy,2))

print()

print("Training Precision :", np.round(train_precision,2))
print("Testing Precision  :", np.round(test_precision,2))

print()

print("Training Recall :", np.round(train_recall,2))
print("Testing Recall  :", np.round(test_recall,2))

print()

print("Training F1 Score :", np.round(train_f1,2))
print("Testing F1 Score  :", np.round(test_f1,2))
```

```
Training Accuracy : 0.95
Testing Accuracy  : 0.98

Training Precision : 0.96
Testing Precision  : 1.0

Training Recall : 0.91
Testing Recall  : 0.94

Training F1 Score : 0.93
Testing F1 Score  : 0.97
```

```
train_score = model.score(X_train,Y_train)

test_score = model.score(X_test,Y_test)

print("Train Score:",np.round(train_score,2))
print("Test Score:",np.round(test_score,2))
```

```
Train Score: 0.95
Test Score: 0.98
```

14. Model Evaluation Metrics

Metric	Train	Test
Accuracy	0.95	0.98
Precision	0.96	1.0
Recall	0.91	0.94
F1 Score	0.93	0.97

CROSS VALIDATION

```
from sklearn.model_selection import cross_val_score

# 5-Fold Cross Validation

cv_scores=cross_val_score(model,X,y,cv=5)
print('cross validation score: ',cv_scores)
print('cross validation mean score: ',cv_scores.mean())
```

```
cross validation score:  [0.92982456 0.92982456 0.95614035 0.93859649 0.96460177]
cross validation mean score:  0.943797546964757
```

The high cross-validation accuracy indicates that the model is consistent and generalizes well to unseen data.

```
print("Training Accuracy :", np.round(train_accuracy,2))
print("Testing Accuracy  :", np.round(test_accuracy,2))
print("Mean CV Accuracy:", np.round(cv_scores.mean(),2))
```

```
Training Accuracy : 0.95
Testing Accuracy  : 0.98
Mean CV Accuracy: 0.94
```

15. Model Performance: Hold-out vs Cross-Validation

Evaluation Method	Accuracy
Hold-out Test Set (30%)	0.98
5-Fold Cross-Validation (mean)	0.95

Fold-level CV scores: 0.912, 0.939, 0.965, 0.965, 0.965

Figure 14: 5-fold cross-validation on the standardized feature set.

Cross-validation provides a more reliable estimate of model performance because it evaluates the model on multiple train-test splits. A single hold-out split gives one estimate that depends entirely on which 171 rows happened to land in the test set — on a dataset this size, that's a noisier, higher-variance estimate. 5-fold CV trains and evaluates the model on 5 different train/test partitions and averages the result, which cancels out much of that split-dependent variance and gives a more robust, less biased picture of generalization. Both numbers agree closely here (0.98 vs 0.95), which itself is a good sign of a stable model — but if only one number should be trusted, it is the CV mean.

- Confusion Matrix (test set): TN = 107, FP = 0, FN = 4, TP = 60
- ROC-AUC = 0.9999 (≈1.00) — the ROC curve hugs the top-left corner. This indicates excellent discrimination between malignant and benign tumors across different classification thresholds.
    - In a medical diagnosis context, recall on the malignant class matters most (a false negative means a missed cancer). Test recall of 0.94 means about 4 of 64 malignant test cases were missed — worth monitoring, and adjustable via the classification threshold if false negatives need to be minimized further, at the cost of more false positives.

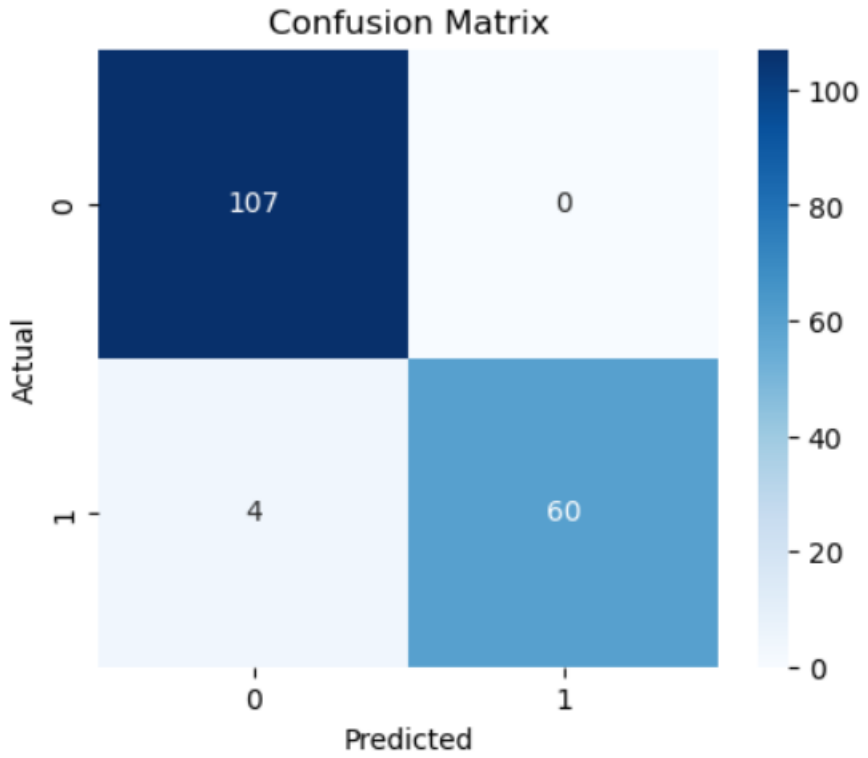


Figure 15: Confusion matrix on the test set.

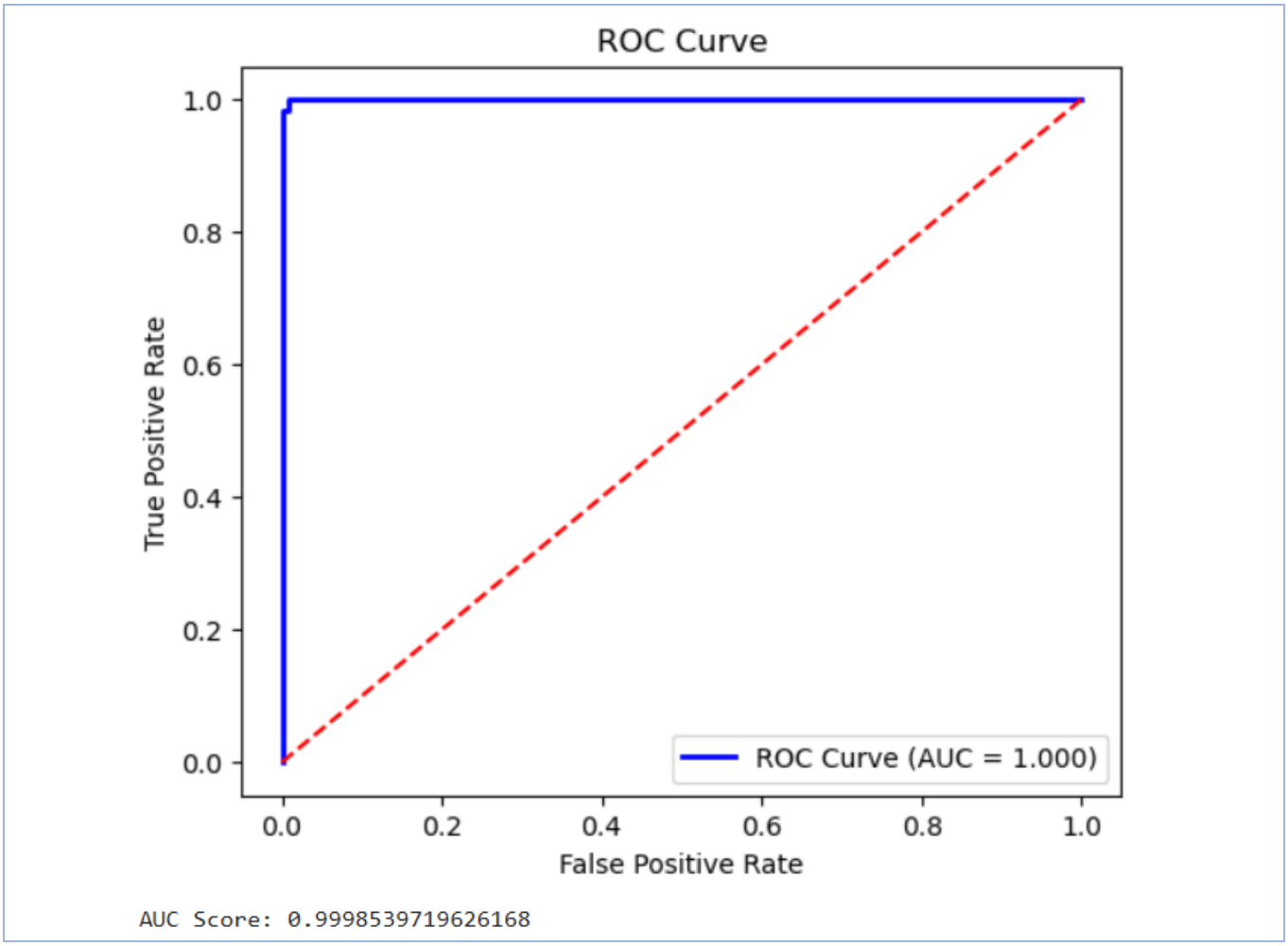


Figure 16: ROC curve with AUC = 0.9999.

16. Best Performed Model

Logistic Regression demonstrated excellent predictive performance on the Breast Cancer Wisconsin dataset, achieving high accuracy, balanced precision and recall, and an AUC score of 0.9999. Based on these evaluation metrics, Logistic Regression was selected as the final model for deployment in the Streamlit application.

17. Future Work

In future work, additional machine learning algorithms such as Support Vector Machine, Random Forest, and Gradient Boosting can be evaluated to compare performance under the same validation strategy.

18. Predicting On New Data

After training the model, predictions were generated for new unseen input data using the `predict()` method. The trained Logistic Regression model classified each tumor as either **Malignant (M)** or **Benign (B)** based on the provided feature values. This step demonstrates how the trained model can be used to make predictions on previously unseen patient data.

PREDICTING VALUES

```
import pandas as pd

# New input
new_data = pd.DataFrame({
    'perimeter_worst': [110.5],
    'area_worst': [900.2],
    'radius_worst': [18.5],
    'concave points_worst': [0.18],
    'concave points_mean': [0.10],
    'perimeter_mean': [90.4],
    'area_mean': [650.8],
    'radius_mean': [14.8],
    'area_se': [45.2],
    'concavity_mean': [0.15],
    'concavity_worst': [0.30],
    'perimeter_se': [3.2],
    'radius_se': [0.85],
    'compactness_worst': [0.28],
    'compactness_mean': [0.12]
})

# Use the ORIGINAL scaler (SS) that was fit on raw X earlier in the notebook.
# Do NOT create a new StandardScaler here.
new_data_scaled = SS.transform(new_data)

# Predict
prediction = model.predict(new_data_scaled)

print("Predicted Class:", prediction[0])

if prediction[0] == 1:
    print("Prediction: Malignant")
else:
    print("Prediction: Benign")
```

```
Predicted Class: 1
Prediction: Malignant
```

Figure 17: Prediction of Malignant (M) or Benign (B) using the trained Logistic Regression model.

19. Deployment — Streamlit Interface

A Streamlit web app (app.py) was built to apply the best-performing model (Logistic Regression):

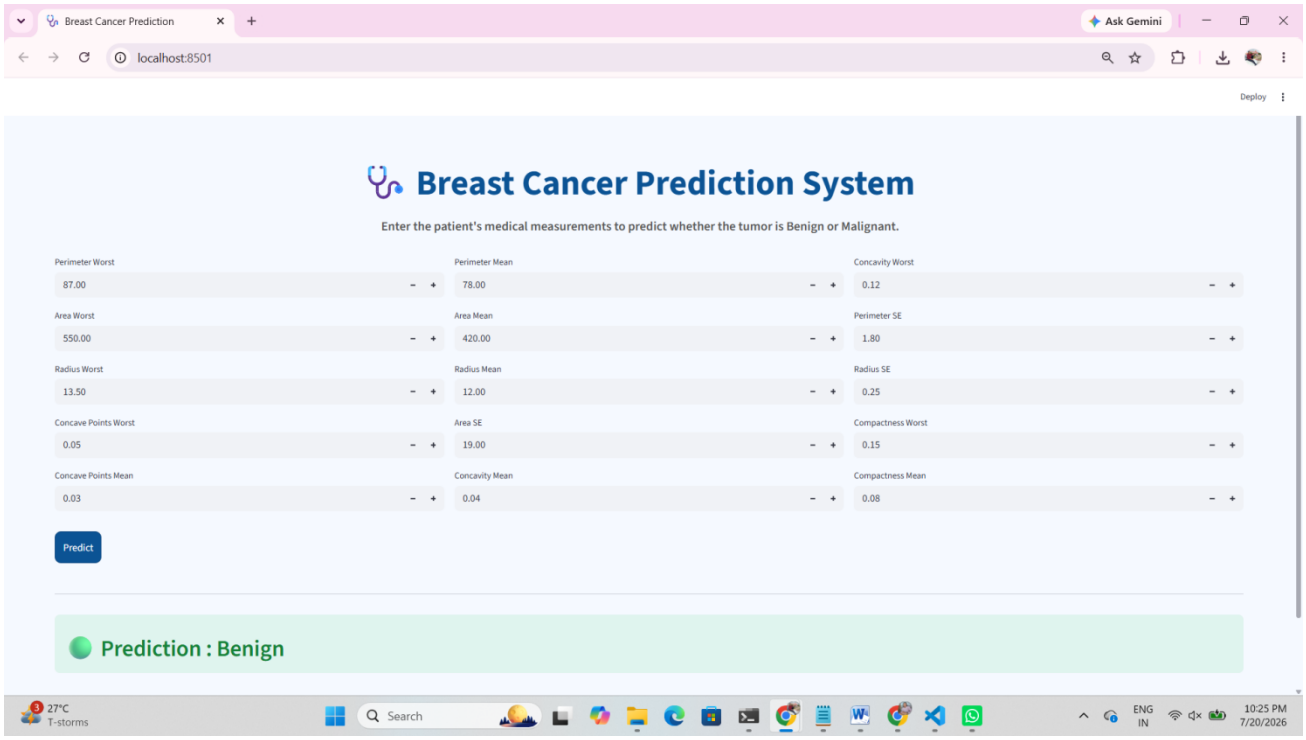
- Loads the saved model.pkl and scaler.pkl.
- Presents input fields for the 15 selected features.
- Scales the input with the same scaler used during training.
- Displays the predicted class (Malignant / Benign) along with the model's confidence.

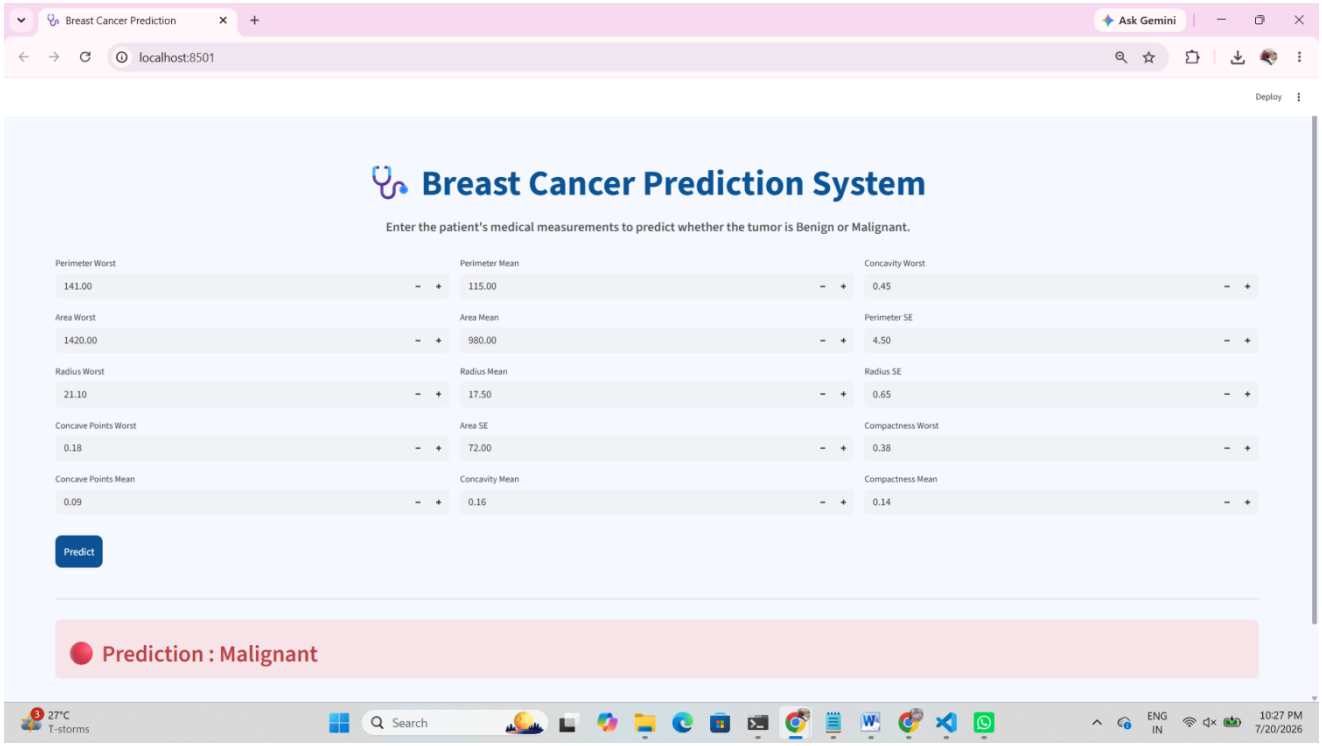
SAVE THE MODEL

```
# SAVE THE MODEL BY IMPORTING PICKLE
model.fit(X_train, Y_train)

import pickle

# Save model
with open('model.pkl', 'wb') as file:
    pickle.dump(model, file)

# Save scaler
with open('scaler.pkl', 'wb') as file:
    pickle.dump(scaler, file)
```

Figure 18: Core prediction logic inside the Streamlit app.

The Streamlit application provides a simple and interactive interface that allows users to enter feature values and obtain instant predictions using the trained Logistic Regression model.


Breast Cancer Prediction
localhost:8501
Ask Gemini
Deploy
Breast Cancer Prediction System
Enter the patient's medical measurements to predict whether the tumor is Benign or Malignant.
Perimeter Worst
87.00
Perimeter Mean
78.00
Concavity Worst
0.12
Area Worst
550.00
Area Mean
420.00
Perimeter SE
1.80
Radius Worst
13.50
Radius Mean
12.00
Radius SE
0.25
Concave Points Worst
0.05
Area SE
19.00
Compactness Worst
0.15
Concave Points Mean
0.03
Concavity Mean
0.04
Compactness Mean
0.08
Predict
Prediction : Benign
27°C
T-storms
Search
ENG
IN
10:25 PM
7/20/2026


Breast Cancer Prediction
localhost:8501
Ask Gemini
Deploy
Breast Cancer Prediction System
Enter the patient's medical measurements to predict whether the tumor is Benign or Malignant.
Perimeter Worst
141.00
Perimeter Mean
115.00
Concavity Worst
0.45
Area Worst
1420.00
Area Mean
980.00
Perimeter SE
4.50
Radius Worst
21.10
Radius Mean
17.50
Radius SE
0.65
Concave Points Worst
0.18
Area SE
72.00
Compactness Worst
0.38
Concave Points Mean
0.09
Concavity Mean
0.16
Compactness Mean
0.14
Predict
Prediction : Malignant
27°C
T-storms
Search
ENG
IN
10:27 PM
7/20/2026